PROJECT-BASED LEARNING

PHASE-III FINAL EVALUATION

Project Title

Team ID: DSCPP-III-2026-T113

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T113
Semester: 3rd
Section: J,K,G
Domain: *E-commerce*

Team Members

1. *Shaurya Bisht – 2510012399*
2. *Mayank Ranjan – 2510011761*
3. *Raj Vardhan – 2510010483*
4. *Aradhay Singh – 2510011780*

Mentor

Mr.Priyanshu Rawat

Interactions: _____

Project Abstract & Problem Statement

Project Abstract

A hyperlocal e-commerce system where neighbourhood shops sell online and local delivery agents deliver.

Problem Statement

Apps like Blinkit and Zepto need their own stores and large networks. Small shops have no online channel, and customers can't easily get home delivery from them.

Target Users / Application

- *Primary users: shopkeepers, customers, delivery agents and an admin.*
- *Application environment: runs locally in a browser at localhost:8080.*
- *Expected practical benefit: shops get online sales, customers get doorstep delivery*

Objectives & Final Deliverables

Finalized Objectives

1. *Shopkeepers manage products, prices and stock.*
2. *Customers browse, compare prices across shops and place orders.*
3. *Shop orders are handled first-come-first-served.*
4. *The nearest free agent is assigned using Dijkstra.*
5. *Customers track the order live and see the bill.*

Final Deliverables

- *working web app*
- *C++ source code*
- *text-file data with demo data*
- *README and report*
- *test scripts.*

Phase-I & Phase-II Feedback Incorporated

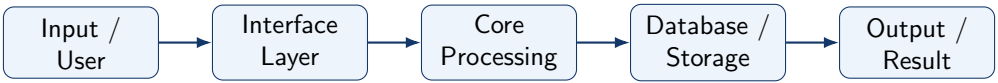
Phase	Feedback / Suggestion	Action Taken
Phase-I	*Keep the scope realistic*	*Modification incorporated*
Phase-I	*Feedback received during Phase-I*	*Modification incorporated*
Phase-II	*Feedback received during Phase-II*	*Modification incorporated*
Phase-II	*Feedback received during Phase-II*	*Modification incorporated*

Final Improvement

Briefly explain how the project improved from the initial proposal to the final implementation.

Brief Discussion: Evaluators should be able to clearly identify how earlier feedback has improved the final project.

Final System Architecture



Major Components

- *web frontend (HTML/CSS/JS),*
- *C++ server with a JSON API,*
- *six C++ modules (Common, Shop, Order, Customer, Delivery agent, Storage)*
- *text-file storage.*

System Flow

- *Input mechanism: forms and buttons.*
- *Processing mechanism: validate, then module logic.*
- *Storage mechanism: save to files after each change.*
- *Output mechanism: pages, order status, ETA and map.*

Final Technology Stack

Software / Technologies

- Programming: *C++ (backend), HTML/CSS/JavaScript (frontend).*
- Framework: *none;own small HTTP server.*
- Database: *text files.*
- Libraries/APIs: *C++ standard library*
- OS/Platform: *Leaflet with OpenStreetMap*

Development Infrastructure

- IDE: *VS Code.*
- Version Control: Git/GitHub
- Testing Tools: *Python API tests, Playwright browser tests.*
- Deployment: *local, via run.sh or run.bat.*
- Other Tools: *none*

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures in C	*Linked list (all orders), linked FIFO queues (shop desk and dispatch), chained hash tables (inventory, product names), arrays, graph with Dijkstra, bubble and insertion sort*	*Shop, Order, Customer, Delivery*
OOPs with C++	*Classes, objects, inheritance (User → Customer, Shopkeeper, DeliveryAgent, Admin), virtual functions*	*All modules*

Module-Wise Final Implementation

Module	Implementation	Responsible	Status
Shop management	*Hash tables, catalogue, add/update/delete/view stock, shop order desk*	*Mayank*	Completed
Order management	*Order queue, linked list, status flow, shop allocation, fees*	*Shaurya*	Completed
Customer	*Cart, search, price comparison by sorting, checkout, order tracking*	*Raj*	Completed
Delivery agent	*Registration, Dijkstra nearest agent, pickup/delivery, live tracking*	*Aradhay*	Completed

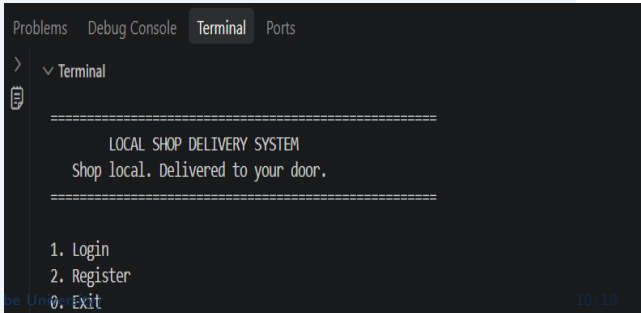
Final Project Completion: *99.9%*

Working System / Prototype Demonstration

Major Working Features

- *The customer picks an area, searches and fills the cart.*
- *Prices are compared and a shop is chosen: nearest, cheapest or best overall.*
- *The order goes to the shop's queue, and the shopkeeper accepts and packs it.*
- *The nearest free agent is found by Dijkstra, then confirms pickup and delivery.*

System Screenshot / Output



Data / Database / Backend Implementation

Data / Database

- *Data: text files in backend/data/ (users, agents, products, shops, inventory, orders, $order_items$, payments, $area_graph$).*
- *Everything is saved after each change and loaded on start.*
- *Validation: isNumber, parseInt and validName reject bad input with a clear error.*

Backend / Core Logic

- *A machine allows only valid moves: PLACED → ACCEPTED → PACKED → AGENT;ASSIGNED → PICKED;UP → DELIVERED(plus CANCELLED and REJECTED).*
- *API: JSON endpoints under /api with error codes 400, 401, 403, 404 and 409.*
- *Dijkstra runs on 8 areas.*
- *Security: login token per session, and a simple password hash limit*

Testing & Validation

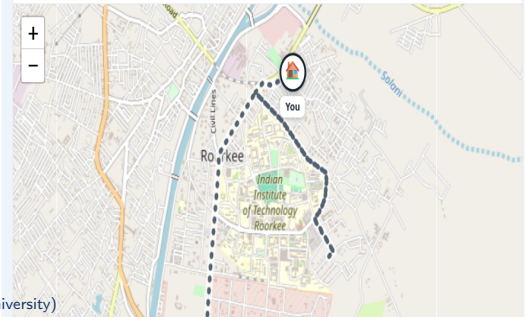
Test Type	Expected Result	Observed Result	Status
Backend API tests	*Valid status flow, correct fees, roles enforced, FIFO desk, no overselling, data survives restart*	*5 suites, 165 checks*	Pass
Integration	*Full order to delivery across all four roles*	*Delivered, bill and money split correct*	Pass
System (browser)	*Every screen of all four roles, phone layout*	*114 checks*	Pass
Input validatio	*Bad input does not crash*	*Error shown, app keeps running*	*Pass*

Results & Performance Analysis

Key Results

- *Worked example (3 km order): items 74 + delivery 35 + small-cart fee 15 + platform fee 5 = 129.*
- *Money split: shop 66.60, rider 28, platform 34.40.*
- *Meaning: the shop is paid for items after commission, the rider is paid for delivery, and the platform keeps the fees.*

Graph / Result Evidence



GitHub Repository & Development Evidence

Repository Details

- Repository: *https://github.com/shauryabisht75-ui/Local-shop-delivery-system*
- Branch: *main*
- README.md: *https://github.com/shauryabisht75-ui/Local-shop-delivery-system-/blob/main/README.md*
- Contributors: *All members*
- Documentation: *Available*

Development Evidence

- *Regular commits*
- *Meaningful commit messages*
- *Individual commits*
- *Latest source code*
- *Complete commit history*

Individual Contribution

Member	Role	Final Technical Contribution	Commits
Shaurya	*Team lead*	*Order queue, linked list status flow, shop allocation and fees, integration and testing*	*XX*
Mayank	*Shop management*	*Product and Shop classes, hash tables, stock add/update/delete/view*	*XX*
Raj	*Customer module*	*Customer class, cart, browse, price comparison by sorting, order tracking*	*XX*
Aradhay	*Delivery agent*	*User and DeliveryAgent classes, registration, Dijkstra, delivery completion, tracking simulation*	*XX*

Challenges, Limitations & Lessons Learned

Challenges & Solutions

- *Finding the nearest agent without GPS–area graph plus Dijkstra.*
- *Bad input crashing the program–validation everywhere.*
- *Two customers buying the last item–stock is reserved at order time.*

Current Limitations

- *Google Maps with a real key was not tested*
- *no rating system*

Technical Learning

- *Dijkstra versus BFS*
- *separating logic from the web layer.*
- *choosing a data structure per feature*

Team Learning

- *Git branches, code review*
- *a weekly check-in.*

Final Outcomes, Applications & Future Scope

Final Outcomes

- *Outcomes: a working order-to-delivery system for four roles, with DSA and OOP demonstrated and tests included.*

Applications

- *Applications: local retail shops selling online, neighbourhood delivery, and delivery agents earning extra income.*

Future Scope

- *Future scope: rating system, real rider GPS, a real payment gateway, a MySQL database, a mobile app.*

Tangible Project Outcome

a working prototype with source code, README and tests on GitHub.

References & Project Resources

References

1. *Introduction to Algorithms*
2. *Stroustrup*
3. *The C++ Programming Language*
4. *Leaflet and OpenStreetMap documentation*

Project Resources

- GitHub: *https://github.com/shauryabisht75-ui/Local-shop-delivery-system*
- Demo: *runs locally at localhost:8080*
- Dataset: *NA*
- Documentation: *README.md.*

Thank You

Questions & Discussion